

HelixConstitution

HelixConstitution

Универзални инжењерски устав који сваки пројекат наслеђује — закон против блефирања, механички спроведен, дељен као један Гит подмодул.

Резиме

HelixConstitution је јединствен, пројектно-агностичан правилник — додаван као Гит подмодул у сваком Helix/vasic-digital пројекту — који кодира непреговорљиву инжењерску дисциплину (анти-блефирање, валидација искључиво на основу доказа, безбедност података и хоста, документација и покривеност тестовима) и преноси је на флоту од преко 140 репозиторијума. Он је управљачка кичма која читавој породици даје кохерентност.

Кратак опис

Универзални, наследиви Constitution испоручен као Гит подмодул. Дефинише обавезна, непреговорљива правила — контролне тачке против блефирања засноване на доказима, имунитет на лажно позитивне резултате, безбедност података и хоста, дисциплину покривености тестовима и документације — која сваки пројекат који га користи аутоматски наслеђује и може да прошири, али никада да ослаби.

Детаљан опис

HelixConstitution је канонски, једини извор истине за инжењерске праксе које деле сви пројекти који се одреде за њега додавањем као Гит подмодула — инжењерски закон, дистрибуиран и верзиониран тачно као код. Његов централни део — Constitution.md — јесте документ од ~1 МБ, непрестано верзиониран, са нумерисаним клаузулама (породица завета §11.4.x, тренутно до §11.4.170) и приручницима за поједине

агенте (CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md), који га увозе референцом тако да људи и сваки CLI агент читају из једног идентичног правилника. Наслеђивање је намерно троструко: универзална основа (овај подмодул), слој пројекта (пројектов сопствени Constitution/ЦЛАУДЕ/АГЕНТС који га проширује) и опциони слој по поддиректоријуму — евалуиран одозго надоле, где пројекат може да *појача* правила, али је архитектонски спречен да их *ослаби*. Резултат је флота од преко 140 репозиторијума који не могу тихо да се удаље једни од других, јер је дисциплина коју деле фиксирана, а не пуко запамћена.

Документ је непопустљиво агностичан према домену: све што именује одређеног добављача, хардверски SKU, порт или верзију библиотеке мора да се помери надоле у сопствени Constitution пројекта који га користи, а универзалност се никада не подразумева — мора да се *заслужи* проласком кроз експлицитни четвороделни тест пре него што се неко правило допусти у основу. Његов филозофски стуб је анти-блефирање, изражен као међусобно повезан низ завета — §1.1 имунитет на лажно позитивне резултате, §11.4 завет о квалитету за крајњег корисника, §11.4.6 забрана нагађања, §11.4.69 таксономија позитивних доказа — чији комбиновани ефекат је једна јасна црвена линија: стандард за испоруку никада није „тестови пролазе“, већ „прави корисник може да користи функцију“, и сваки позитиван резултат мора да наведе ухваћене физичке доказе или се не рачуна. Пратећи submodules-catalogue.md (142 репоа) претвара питање „да ли већ имамо нешто што ово ради?“ у рефлекс „прво каталог, па тек онда прошири, не имплементирај поново“ пре него што се напише иједна нова линија кода. Помоћни скриптови проналазе подмодул са било које дубине угнежђености и распростиру сваки комит на четири независна Гит провајдера, тако да је једини ауторитативни правилник уједно и немогућ за губљење.

Зашто смо га изградили

Више великих апликација за производе и десетине одвојених, поново употребљивих потмодула, које је креирао исти власник, стално су поново изводили исте, мукотрпно стечене правила — и стално наилазили на исту врсту грешака: тестове и извештаје о статусу који тврде да је све у реду док је функција заправо неисправна за крајњег корисника („лажни ПАСС“ и „лажни ФАИЛ“). Свака форензичка референца у Constitution бележи стваран инцидент (нпр. лажни ПАСС рутирања звука D3 од 20. маја 2026, где је валидација била зелена упркос празном пољу „Кодек у употреби“, или циновски UI дугмић од 25. јуна 2026. који је прошао тестове једнакости токена док је стварни екран био покварен). Constitution постоји да целу ову

класу непоштеног успеха учини механички немогућом — једном, универзално — како дисциплина не би клизила између пројеката нити била тихо заборављена.

Зашто је револуционарно

Он претвара инжењерску културу из документације-коју-се-надају-да-ће-пратити у наслеђено, верзирано, механички наметнуто правило — разлику између водича за стил и компајлера. Једно ажурирање потмодула истовремено унапређује правила за цео систем, атомски и са могућношћу праћења. Један анти-лажни завет је *загарантовано* присутан у сваком репозиторијуму који га користи, не захваљујући поверењу, већ конструкцији: пропагациони механизам буквално претражује клаузулу по броју у целом систему, а упарени мутациони тест доказује да сам механизам не лаже — тако да је чак и наметање правила наметнуто. Управљање престаје да буде жеља записана на викију који нико не чита и постаје проверљива, тестабилна чињеница на коју можете упутити ЦИ посао.

Шта је иновативно

- **Constitution као потмодул** — инжењерски закон дистрибуиран и верзиран тачно као код, са намерним ознакама у стилу `v1.0.0` и фиксирањем верзија по пројекту, тако да сваки репозиторијум *тачно* зна коју ревизију закона мора да поштује.
- **Анти-лаж као првокласна, форензичка доктрина** — свака клаузула води порекло од буквалне наредбе оператера, а често и од тачног инцидента из стварног света који ју је мотивисао, тако да збирка правила чита као судска пракса, а не као мишљење.
- **Meta-тестирање самих правила (§1.1)** — сваки механизам је упарен са мутацијом која мора да промени ПАСС→ФАИЛ, тако да „механизам није лажан” није само тврдња, већ се доказује при сваком покретању; механизам који никада не може да закажује сматра се горим од одсуства механизма.
- **Заслужена универзалност** — експлицитан тест у четири дела одлучује да ли је правило заиста универзално или само специфично за пројекат, чиме се база одржава витком, преносивом и ослобођеном утицаја појединих добављача.

Како се користи у свим производима (моћи које пружа)

Као **обавезни стуб управљања**, HelixConstitution није документ који породица консултује — то је носећа конструкција на којој је породица изграђена:

- **Стуб управљања:** сваки пројекат Helix/vasic-digital додаје га као потмодул и увози из CLAUDE.md / AGENTS.md / QWEN.md / сопственог Constitution.md; правила се примењују безусловно, од првог комита, без могућности изузетка по пројекту.
- **Механизми и обавезе:** дефинише четворослојни модел покривености — присутан-у-коду, преживи-компилацију, понаша-се-у-рунitime-у, механизам-није-лажан — који функција мора да прође на сва четири нивоа пре него што се сматра завршеном, уз растући списак именованих обавеза: руковање акредитивима (§11.4.10), увек-синхронизована документација (§11.4.60), обавеза потмодула за контејнере (§11.4.76), ЦодеГрапх (§11.4.78), обавезно покривање типова тестова (§11.4.169) и још много тога.
- **Пропагација:** CM-COVENANT-114-NNN-PROPAGATION механизми потврђују да је *буквалан* текст клаузуле присутан у свим репозиторијумима који га користе, тако да се обавеза не може тихо изоставити у неком делу система; неусаглашеност је тврда блокада издања без могућности заобилажења.
- **Откривање:** submodules-catalogue.md претвара питање „да ли већ имамо нешто што ради X?” у одговор на први поглед, пре него што се било који нови модул иницијализује, спречавајући дуплирање на извору.
- **Конзистентност AI агената:** исти закон се идентично изражава сваком агенту CLI (Claude Цоде, Codex/Cursor/Aider/OpenCode/Crush/Kimi преко АГЕНТС.мд, Qwen Цоде преко QWEN.мд), тако да без обзира који алат додирне код, он поштује један те исти завет.

Највећи технички изазови и како смо их решили

- **Локализација подмодула из произвољне угњеждене дубине** — правило закопано три подмодула дубоко и даље мора пронаћи закон без обзира на то где се налази → find_constitution.sh пење се кроз родитељске директоријуме и рекурзивно прати показивач гит суперпројекта, поштујући надјачавање CONSTITUTION_DIR и два подржана распореда (constitution/, submodules/constitution/), тако да је

разређење детерминистичко без обзира на дубину угњежђености.

- **Одржавање једног репозиторијума као ауторитативног на четири Гит провајдера** — огледала су бескорисна ако се разилазе → `install_upstreams.sh` чита декларативне `Upstreams/*.sh` удаљене репозиторијуме и конфигурише `origin` са више URL-ова за слање, тако да један `git push` атомски шаље промене на GitHub (примарни), GitLab, GitFlic и GitVerse, а ниједно огледало не може заостати.
- **Спречавање бујања правила / цурења пројекта у универзалну базу** — свако примамљиво „само додај овде“ нарушава преносивост → примењује се четвороделни тест стечене универзалности плус класификација универзално-пројектно по §11.4.17 на свако ново правило, што пројектно-специфичне проблеме враћа назад у пројектни слој где им је место.
- **Доказивање да механизам наслеђивања стварно функционише** — капија коју никада не видите да отказује је капија којој не можете веровати → `meta_test_inheritance.sh`, мета-тест чувар, намерно брише сидро §11.4 и проверава да ли капија то хвата, тако да се сам механизам провере непрестано поново верификује против тихог отказивања.

Технолошки стек

- **Наслеђивање Гит подмодула** — *зашто*: Гит подмодули су једини механизам који омогућава да збирка правила буде ауторитативна и верзијски закључана по потрошачу, надограђена експлицитним, прегледљивим ажурирањем уместо тихим копирањем и лепљењем; *како*: пројекти који користе збирку додају подмодул и `@import-ују` његове агентске фајлове, а три слоја се евалуирају одозго надоле са строгим уговором „проширује, не слаби“ на свакој граници.
- **`find_constitution.sh`** — *зашто*: правила су бескорисна ако их дубоко угњежђени код не може поуздано пронаћи, а хардкодирање путева би отказало чим би пројекат променио структуру; *како*: пењање кроз родитељске директоријуме плус рекурзија `git rev-parse --show-superproject-working-tree`, подржано надјачавањем `CONSTITUTION_DIR`, разрешава оба подржана распореда.
- **`install_upstreams.sh` + `Upstreams/`** — *зашто*: четворострука редунданса је стварна само ако не захтева додатни напор за одржавање, иначе огледала пропадају; *како*: декларативни `.sh` фајлови по удаљеном репозиторијуму се материјализују у један мулти-URL `origin`, спајајући четири слања у једно.

- **Meta-тестови мутација по §1.1** — *зашто*: капија која никада не отказује је гора од непостојеће јер ствара лажну сигурност; *како*: свака капија је упарена са мутацијом брисања/преименовања која мора да промени статус из ПРОЛАЗИ у ПАДА, а затим се враћа у првобитно стање, тако да свака капија доказује да и даље „угристи” при сваком покретању.
- **Капије пропагације (CM-COVENANT-114-NNN-PROPAGATION)** — *зашто*: уговор је универзално примењив само ако је проверљиво присутан у сваком потрошачу, а не само у водећем репозиторијуму; *како*: буквална претрага броја клаузуле по потрошачима, подржана упареним §1.1 мета-тестом мутације који доказује да сама провера пропагације може да откаже.
- **submodules-catalogue.md (§11.4.74)** — *зашто*: најбржи начин да се наруши дисциплина против дуплирања је да не знате шта већ поседујете; *како*: инвентар од 142 репозиторијума груписаних по могућностима, са провером каталога забележеном у трагачу *пре* него што се ишта ново постави.
- **Извоз у више формата** — *зашто*: исти закон мора бити једнако доступан људима који га читају, алатима који га обрађују и архивама које га чувају; *како*: сваки канонски документ се извози као .md / .html / .pdf / .docx из једног извора.

Напомене о статусу и искрености

- **Статус: испоручено.** Активно верзионирано и користи се као потмодул широм флоте (јавни канонски и миррор репозиторији).
- **Лиценца: треба утврдити** — није експлицитно наведена у прегледаном изворном материјалу; потврдити према фајлу ЛИЦЕНСЕ у репозиторијуму пре објављивања.
- Додатна узводна огледала: GitLab helixdevelopment1/helixconstitution, GitFlic helixdevelopment/helixconstitution, GitVerse helixdevelopment/HelixConstitution.

Приоритетни ниво: Helix-примарни — обавезан управљачки стуб на којем почива изградња свега у породици Helix.